in particular; one of the design goals has been that there should be little or no benefit to be gained from using the lower-level APIs directly, where the same facility is to be provided. The library has therefore been designed to allow for efficient implementation (with a very low abstraction penalty) on most major platforms.

Another goal of the C++ Standards Committee has been to ensure that C++ provides sufficient low-level facilities for those wishing to work close to the metal for the ultimate performance. To this end, along with the new memory model comes a comprehensive atomic operations library for direct control over individual bits and bytes and the inter-thread synchronization and visibility of any changes. These atomic types and the corresponding operations can now be used in many places where developers would previously have chosen to drop down to platform-specific assembly language. Code using the new standard types and operations is thus more portable and easier to maintain.

The C++ Standard Library also provides higher-level abstractions and facilities that make writing multithreaded code easier and less error prone. Sometimes the use of these facilities does come with a performance cost because of the additional code that must be executed. But this performance cost doesn't necessarily imply a higher abstraction penalty; in general the cost is no higher than would be incurred by writing equivalent functionality by hand, and the compiler may well inline much of the additional code anyway.

In some cases, the high-level facilities provide additional functionality beyond what may be required for a specific use. Most of the time this is not an issue: you don't pay for what you don't use. On rare occasions, this unused functionality will impact the performance of other code. If you're aiming for performance and the cost is too high, you may be better off handcrafting the desired functionality from lower-level facilities. In the vast majority of cases, the additional complexity and chance of errors far outweigh the potential benefits from a small performance gain. Even if profiling *does* demonstrate that the bottleneck is in the C++ Standard Library facilities, it may be due to poor application design rather than a poor library implementation. For example, if too many threads are